U24AI047 AI LAB PRACTICAL-14

1. Write a program to solve the questions using Forward Chaining.

a. Knowledge Base:

P → Q

L ∧ M → P

A ∧ B → L

A

B

M

Conclusion:

Q

b. Knowledge Base:

A → B

B → C

C → D

A

E

D ∧ E → F

Conclusion: F

```
def pl_fc_entails(rules, known_facts, q):
    # Initialize count table for each rule's premises
    count = {i: len(premise) for i, (premise, conclusion) in
enumerate(rules)}

    # Gather all unique symbols and track inference status
    symbols = set(known_facts)
    for premise, conclusion in rules:
        symbols.update(premise)
        symbols.add(conclusion)
```

```python
    inferred = {s: False for s in symbols}

    queue = list(known_facts)

    print(f"--- Starting PL-FC-ENTAILS? ---\nInitial Queue: 
{queue}\nTarget: {q}\n")

    while queue:
        p = queue.pop(0)

        if p == q:
            print(f"\nSuccess! Found '{q}'.")
            return True

        if not inferred.get(p, False):
            inferred[p] = True
            print(f"Processing: {p}")

            for i, (premise, conclusion) in enumerate(rules):
                if p in premise:
                    count[i] -= 1
                    if count[i] == 0:
                        print(f"  -> Rule {i} ({premise} -> 
{conclusion}) satisfied. Queuing '{conclusion}'.")
                        queue.append(conclusion)

    print(f"\nFailure. '{q}' not entailed.")
    return False

# Problem A
print("=== PROBLEM A ===")
rules_a = [(['P'], 'Q'), (['L', 'M'], 'P'), (['A', 'B'], 'L')]
facts_a = ['A', 'B', 'M']
pl_fc_entails(rules_a, facts_a, 'Q')

print("\n")

# Problem B
print("=== PROBLEM B ===")
rules_b = [(['A'], 'B'), (['B'], 'C'), (['C'], 'D'), (['D', 
'E'], 'F')]
facts_b = ['A', 'E']
```

```
pl_fc_entails(rules_b, facts_b, 'F')
```

```
param@GREYRAT MINGW64 /c/SVNIT PRACTICALS/AI-lab
$ python -u "c:\SVNIT PRACTICALS\AI-lab\LAB-14\q1.py"
=== PROBLEM A ===
--- Starting PL-FC-ENTAILS? ---
Initial Queue: ['A', 'B', 'M']
Target: Q

Processing: A
Processing: B
  -> Rule 2 (['A', 'B'] -> L) satisfied. Queuing 'L'.
Processing: M
Processing: L
  -> Rule 1 (['L', 'M'] -> P) satisfied. Queuing 'P'.
Processing: P
  -> Rule 0 (['P'] -> Q) satisfied. Queuing 'Q'.

Success! Found 'Q'.


=== PROBLEM B ===
--- Starting PL-FC-ENTAILS? ---
Initial Queue: ['A', 'E']
Target: F

Processing: A
  -> Rule 0 (['A'] -> B) satisfied. Queuing 'B'.
Processing: E
Processing: B
  -> Rule 1 (['B'] -> C) satisfied. Queuing 'C'.
Processing: C
  -> Rule 2 (['C'] -> D) satisfied. Queuing 'D'.
Processing: D
  -> Rule 3 (['D', 'E'] -> F) satisfied. Queuing 'F'.
```

2. Write a program to solve the questions using Backward Chaining.

a. Knowledge Base:

P → Q

R → Q

A → P

B → R

A

B

Conclusion:

Q

b. Knowledge Base:

A → B

B ∧ C → D

E → C

A

E

Conclusion:

D

```
def pl_bc_entails(rules, known_facts, query):
    def prove(q, path):
        indent = "  " * len(path)
        print(f"{indent}[?] Attempting to prove: '{q}'")

        if q in known_facts:
            print(f"{indent}[+] Success: '{q}' is a known
fact.")
            return True

        if q in path:
            print(f"{indent}[-] Failure: Cycle detected for
'{q}'.")
            return False

        for premises, conclusion in rules:
            if conclusion == q:
                print(f"{indent}[*] Found relevant rule:
{premises} -> {conclusion}")

                all_premises_proved = True
                for p in premises:
```

```
                    if not prove(p, path + [q]):
                        all_premises_proved = False
                        print(f"{indent}[-] Premise '{p}'
failed for rule {premises} -> {conclusion}")
                        break

                if all_premises_proved:
                    print(f"{indent}[+] Success: Proved '{q}'
via rule {premises} -> {conclusion}")
                    return True

        print(f"{indent}[-] Failure: Exhausted all options for
'{q}'.")
        return False

    print(f"--- Starting PL-BC-ENTAILS? ---\nTarget Query:
{query}")
    result = prove(query, [])
    print(f"\nFinal Result: {'SUCCESS' if result else
'FAILURE'}\n")
    return result

# Problem A
print("=== PROBLEM A ===")
rules_a = [(['P'], 'Q'), (['R'], 'Q'), (['A'], 'P'), (['B'],
'R')]
facts_a = ['A', 'B']
pl_bc_entails(rules_a, facts_a, 'Q')

# Problem B
print("=== PROBLEM B ===")
rules_b = [(['A'], 'B'), (['B', 'C'], 'D'), (['E'], 'C')]
facts_b = ['A', 'E']
pl_bc_entails(rules_b, facts_b, 'D')
```

```
param@GREYRAT MINGW64 /c/SVNIT PRACTICALS/AI-lab
$ python -u "c:\SVNIT PRACTICALS\AI-lab\LAB-14\q2.py"
=== PROBLEM A ===
--- Starting PL-BC-ENTAILS? ---
Target Query: Q
[?] Attempting to prove: 'Q'
[*] Found relevant rule: ['P'] -> Q
  [?] Attempting to prove: 'P'
  [*] Found relevant rule: ['A'] -> P
    [?] Attempting to prove: 'A'
    [+] Success: 'A' is a known fact.
  [+] Success: Proved 'P' via rule ['A'] -> P
[+] Success: Proved 'Q' via rule ['P'] -> Q

Final Result: SUCCESS

=== PROBLEM B ===
--- Starting PL-BC-ENTAILS? ---
Target Query: D
[?] Attempting to prove: 'D'
[*] Found relevant rule: ['B', 'C'] -> D
  [?] Attempting to prove: 'B'
  [*] Found relevant rule: ['A'] -> B
    [?] Attempting to prove: 'A'
    [+] Success: 'A' is a known fact.
  [+] Success: Proved 'B' via rule ['A'] -> B
  [?] Attempting to prove: 'C'
```

3. Write a program to solve the questions using the method Resolution and proof of

refutation/contradiction.

a. Knowledge Base:

P v Q

P → R

Q → S

R → S

Conclusion

S

b. Knowledge Base:

P → Q

Q → R

S → ¬R

P

Conclusion:

S

```
def negate(literal):
    return literal[1:] if literal.startswith('~') else '~' +
literal

def resolve(clause1, clause2):
    resolvents = set()
    for l1 in clause1:
        for l2 in clause2:
            if l1 == negate(l2):
                new_clause = (clause1 - {l1}) | (clause2 -
{l2})
                if not any(negate(lit) in new_clause for lit
in new_clause):
                    resolvents.add(frozenset(new_clause))
    return resolvents

def pl_resolution(kb_clauses, query):
    negated_query = frozenset([negate(query)])
    clauses = set(kb_clauses)
    clauses.add(negated_query)

    print(f"Initial Clauses (KB + Negated Query):")
    for c in clauses: print(f"  {set(c)}")
    print("-" * 30)

    new_clauses = set()

    while True:
        n = len(clauses)
        clauses_list = list(clauses)

        for i in range(n):
```

```
            for j in range(i + 1, n):
                resolvents = resolve(clauses_list[i], 
clauses_list[j])

                if frozenset() in resolvents:
                    print(f"[*] Contradiction found! Resolved 
{set(clauses_list[i])} and {set(clauses_list[j])} -> Empty 
Clause")
                    return True

                new_clauses.update(resolvents)

        if new_clauses.issubset(clauses):
            print("[-] No new clauses can be generated. No 
contradiction exists.")
            return False

        clauses.update(new_clauses)


print("=== SOLVING PROBLEM A ===")
kb_a = [
    frozenset(['P', 'Q']),
    frozenset(['~P', 'R']),
    frozenset(['~Q', 'S']),
    frozenset(['~R', 'S'])
]
query_a = 'S'

result_a = pl_resolution(kb_a, query_a)
print(f"Final Conclusion for A: The query '{query_a}' is 
{'PROVEN' if result_a else 'NOT PROVEN'}.\n")


print("=== SOLVING PROBLEM B ===")
kb_b = [
    frozenset(['~P', 'Q']),
    frozenset(['~Q', 'R']),
    frozenset(['~S', '~R']),
    frozenset(['P'])
]
query_b = 'S'
```

```python
result_b = pl_resolution(kb_b, query_b)
print(f"Final Conclusion for B: The query '{query_b}' is
{'PROVEN' if result_b else 'NOT PROVEN'}.")
```

```
param@GREYRAT MINGW64 /c/SVNIT PRACTICALS/AI-lab
$ python -u "c:\SVNIT PRACTICALS\AI-lab\LAB-14\q3.py"
=== SOLVING PROBLEM A ===
Initial Clauses (KB + Negated Query):
  {'~S'}
  {'S', '~Q'}
  {'Q', 'P'}
  {'S', '~R'}
  {'R', '~P'}
------------------------------
[*] Contradiction found! Resolved {'~P'} and {'P'} -> Empty Clause
Final Conclusion for A: The query 'S' is PROVEN.

=== SOLVING PROBLEM B ===
Initial Clauses (KB + Negated Query):
  {'~S'}
  {'R', '~Q'}
  {'P'}
  {'~R', '~S'}
  {'Q', '~P'}
------------------------------
[-] No new clauses can be generated. No contradiction exists.
Final Conclusion for B: The query 'S' is NOT PROVEN.
```